

- Why LinkedIn Created Kafka – And Why It's Moving to NorthGuard
 - Why LinkedIn Created Kafka
 - Cracks at Hyperscale: Kafka's Limitations
 - Key Limitations of Kafka at LinkedIn Scale:
 - NorthGuard (and XInfa): Kafka 2.0 - Rethinking distributed log storage
 - How NorthGuard Overcomes Kafka limitations:
 - 📌 Partitioning
 - ⚙️ Cluster Management
 - 💾 Storage Model
 - 🔁 Consistency
 - ⚡ Workload Tolerance
 - NorthGuard's Data Model
 - XInfa - A Virtualization Layer
 - Summary
 - Lessons Beyond LinkedIn
 - 📖 Further Reading
 - 📖 Background on Kafka's Creation
 - 📖 LinkedIn Engineering Blog
 - 🧩 Technical Deep Dives
 - 📖 Industry Media Coverage
 - 📅 Coming Up in Future Editions
 - 🔔 Closing Thought

Beyond the Stack – Edition #18

Why LinkedIn Created Kafka – And Why It's Moving to NorthGuard

This edition dives into a question many of you have been asking in the **Beyond the Stack** community: **Why did LinkedIn, the birthplace of Kafka, feel the need to build NorthGuard?**

Before we dive in, a **big thank you** for the incredible engagement on my last edition. Your comments, shares, and DMs showed me how deeply this community values exploring the 'why' behind the systems we use every day.

So let's unpack the story—starting with why LinkedIn created Kafka in the first place

Why LinkedIn Created Kafka

When LinkedIn first conceived **Apache Kafka**, it wasn't trying to create the world's most popular streaming platform. It was solving a painful, internal problem.

Back in the early 2010s, LinkedIn's data infrastructure was bursting at the seams. Activity data was being shoveled through **batch XML pipelines** into warehouses, while monitoring data lived in a separate, siloed real-time system. The pain points were everywhere:

- Dozens of brittle XML schemas impossible to scale
- Long delays before data was available
- Siloed systems that couldn't be combined for richer insights
- A centralized data team that had become a bottleneck

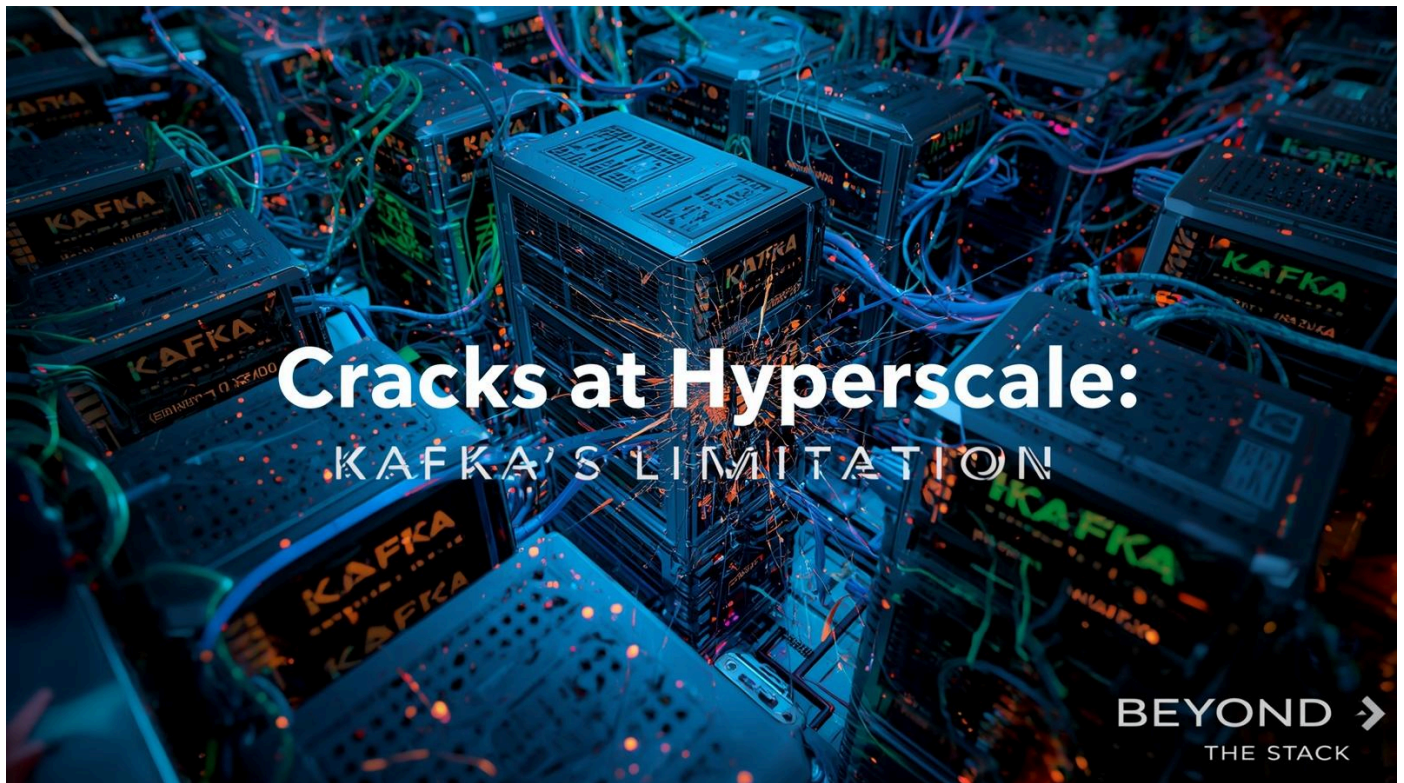
What LinkedIn needed was a **scalable, real-time, durable, schema-governed backbone** —something that allowed any system to publish data and any system to consume it, decoupled and at scale. That's how **Kafka** was born.

Thus, Kafka was developed to:

- Provide durability, horizontal scalability, real-time streaming via its log abstraction.
- Enable decoupling of producers and consumers with retention-based buffering.
- Support high fan-out, real-time data availability.
- Facilitate schema-driven development (with Avro, schema registry, versioning, compatibility enforcement) to enable clean integration across systems.

With log abstraction for durability, Avro + Schema Registry for schema governance, and retention buffering for replay, Kafka unified LinkedIn's batch and stream worlds becoming the **central nervous system of LinkedIn** — and eventually, the world.

Cracks at Hyperscale: Kafka's Limitations



Fast-forward to today. LinkedIn runs Kafka at a mind-boggling scale:

- **32 trillion records/day**
- **17 PB/day throughput**
- **~400,000 topics across 150 clusters**
- **10,000+ machines**

And At this scale, **Kafka's original design assumptions were tested like never before** — forcing LinkedIn to rewrite the rules.

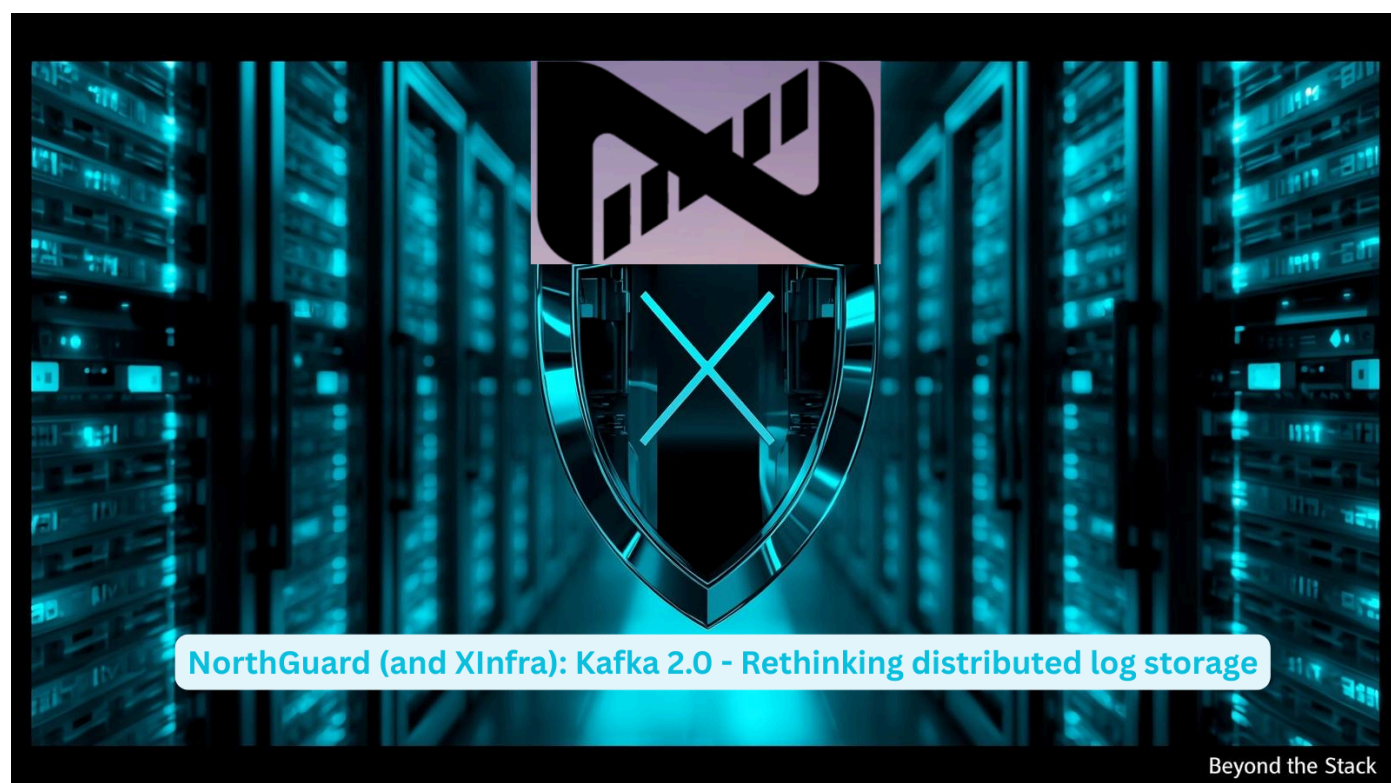
Key Limitations of Kafka at LinkedIn Scale:

- **Scalability Hits** : Metadata growth and cluster fragmentation meant Kafka couldn't manage cluster scaling without manual sharding and spinning up new clusters
- **Operational Complexity** : Load balancing, broker placement, and cluster operations became overwhelming. LinkedIn needed an entire ecosystem just to manage Kafka clusters
- **Partition-Based Constraints** : Partitions were heavy units for replication, limiting flexibility for rebalancing, increasing downtime, and constraining availability and consistency

- **Durability & Consistency** : Kafka's eventual consistency and replication approach weren't robust enough for critical workloads at hyperscale

In short: Kafka worked wonders, but at LinkedIn's hyperscale, its design was showing limits.

NorthGuard (and XInfra): Kafka 2.0 - Rethinking distributed log storage



To address these limitations, LinkedIn built a next-generation system: **NorthGuard**.

NorthGuard is a new internally built log storage system, accompanied by **XInfra** , a virtualization layer to ease migration and compatibility

It's not an open-source Kafka fork — but a complete architectural evolution inspired by Kafka's principles, optimized for modern demands.

NorthGuard represents a fundamental architectural departure from Kafka .

How NorthGuard Overcomes Kafka limitations:

Partitioning

- **Kafka:** Kafka relies on partition-based scaling, where partition counts are pre-decided and rebalancing is heavyweight.
- **NorthGuard:** In contrast, NorthGuard introduces **automatic sharding** that handles both data and metadata, removing the burden of manual configuration.

Cluster Management

- **Kafka:** Scaling a Kafka cluster requires careful planning and manual intervention — adding brokers, triggering rebalances, and monitoring skewed partitions.
- **NorthGuard:** NorthGuard shines here with **self-balancing and self-healing** capabilities. It adapts to changing loads and failures without human intervention.

Storage Model

- **Kafka:** Kafka uses log segments per partition. While effective, it can lead to uneven disk usage and maintenance challenges over time.
- **NorthGuard:** Instead of isolated log segments, NorthGuard employs **log striping across disks** , optimizing performance and capacity utilization.

Consistency

- **Kafka:** Kafka replication ensures durability, but gaps can occur until followers fully sync—insufficient for some mission-critical workloads.
- **NorthGuard:** Aims for **stronger consistency guarantees** , making it better suited for use cases where correctness is critical.

Workload Tolerance

- **Kafka:** Hotspots (overloaded partitions or brokers) are usually resolved manually, often leading to delays or service degradation.
- **NorthGuard:** Designed to prevent hotspots entirely through **adaptive workload distribution** , ensuring better throughput and efficiency.

NorthGuard's Data Model

One of the biggest shifts in NorthGuard is its data model.

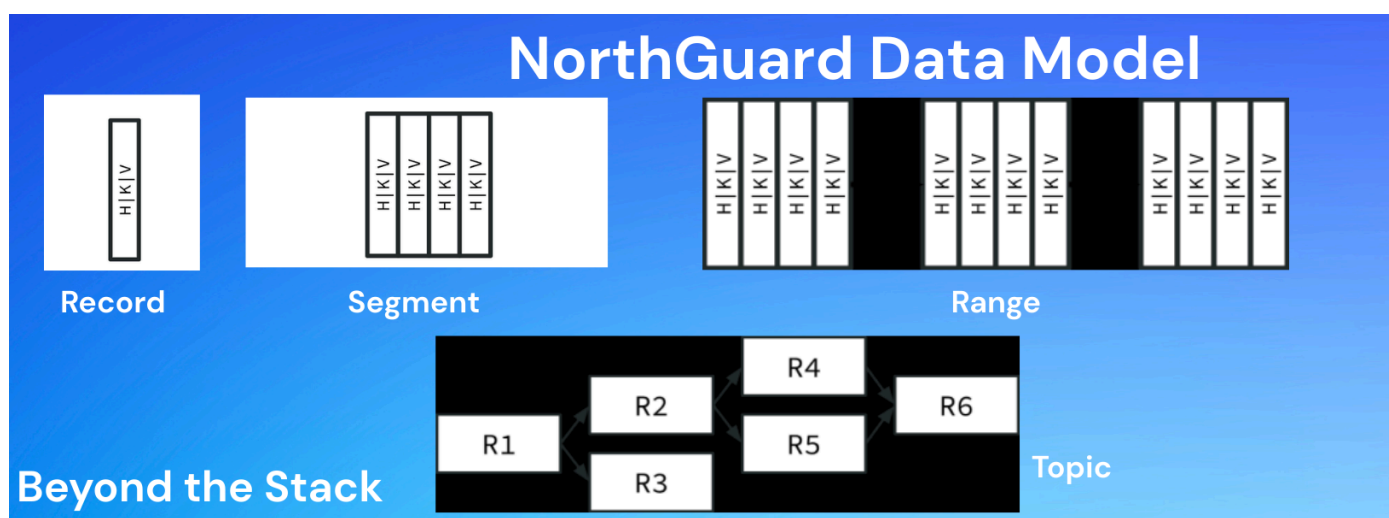
Kafka relies on **partitions** as the core unit of scaling and replication—powerful, but heavyweight and painful to rebalance at hyperscale.

NorthGuard takes a different path. It breaks data into **smaller, flexible segments** that are grouped into ranges. These ranges can be split or merged dynamically, giving NorthGuard far more agility in balancing load and scaling capacity.

In short:

- **Kafka = partitions (heavy, rigid)**
- **NorthGuard = segments + ranges (lightweight, flexible)**

This change under the hood is what enables NorthGuard’s stronger consistency, self-balancing operations, and “lights-out” scalability.



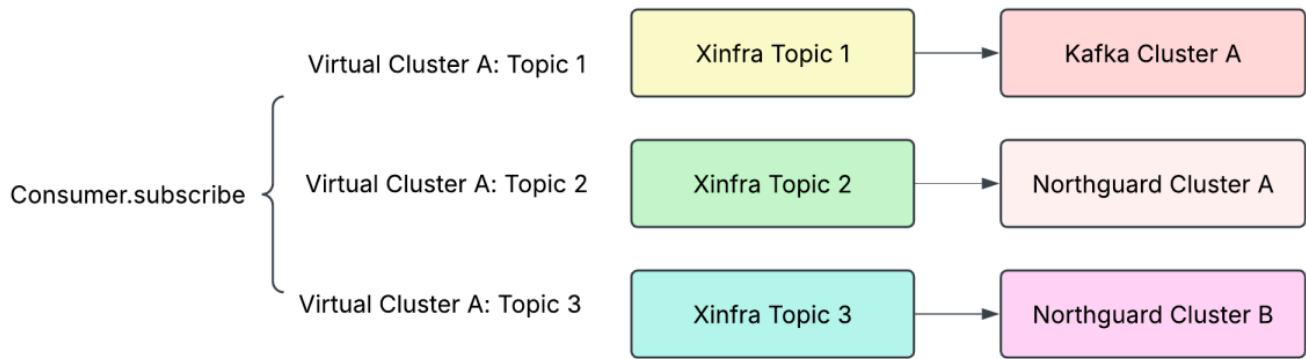
XInfra - A Virtualization Layer

And because you can’t just flip a switch on thousands of topics, LinkedIn built **XInfra**, a virtualization layer.

XInfra (pronounced as ZIN-frah) is a virtualized Pub/Sub layer supporting both NorthGuard and Kafka.

XInfra provides:

- **Virtual Topics** – Apps don’t care if data is on Kafka or NorthGuard.
- **Dual-Writes** – Safe, ordered migration with rollback capability.
- **Zero App Changes** – Over 90% of LinkedIn apps already run on XInfra clients.



Summary

Here’s a summary table of where Kafka hit bottlenecks at hyperscale—and how NorthGuard addresses them.

Area	Original Kafka Design at LinkedIn	Limitations Observed Over Time	NorthGuard & Xinfra Enhancements
Motivation	Batch & real-time pipelines too manual/isolated	Scalability, consistency, schema issues, operations complexity	Segment-based, schema-driven, decoupled pipelines
Replication Unit	Partitions	Heavyweight, inflexible, tedious rebalances	Segments (~1GB/hour), lightweight replication, auto balancing
Metadata Handling	Single controller	Bottlenecks at hyperscale	Sharded metadata via vnodes + Raft
Consistency/Durability	Eventual consistency	Insufficient for critical systems	Strong fsync guarantees, robust replication
Operations	Manual scaling & cluster	High operational burden	Self-balancing, lights-out ops, high

Area	Original Kafka Design at LinkedIn	Limitations Observed Over Time	NorthGuard & XInfa Enhancements
	management		throughput/low latency
Migration	No easy upgrade path	Hard to migrate systems without downtime	XInfa virtualization with dual-writes, transparent migration

Lessons Beyond LinkedIn

Kafka was created to unify batch and stream data pipelines. NorthGuard was created to solve Kafka’s limits at extreme scale. Together, they tell a powerful story:

- **Every abstraction has limits** —at a certain scale, new primitives are needed.
- **Durability and consistency matter more at scale** —eventual consistency isn’t always enough.
- **Virtualization layers (like XInfa) ease disruptive migrations** —a playbook worth remembering.

For most organizations, **Kafka remains more than enough** . But LinkedIn’s journey shows us how systems evolve when scale forces a rethink of the fundamentals.

Further Reading

Want to dive deeper into Kafka’s origins and LinkedIn’s evolution toward NorthGuard? Here are some curated references:

Background on Kafka’s Creation

- [Why Was Apache Kafka Created? – Stanislav Kozlovski](#)

Explains why LinkedIn built Kafka in the first place—solving brittle XML pipelines, schema chaos, and batch + real-time silos.



LinkedIn Engineering Blog

- [Introducing NorthGuard and XInfra – LinkedIn Blog](#)

The official post from LinkedIn engineers, detailing NorthGuard's design and XInfra's role in seamless migration.



Technical Deep Dives

- [LinkedIn Introduces NorthGuard and XInfra – InfoQ](#)

Covers segment-level replication, Raft-backed metadata, and migration mechanisms.

- [LinkedIn Infrastructure Revolution: NorthGuard and XInfra – WebMobix](#)

Explains how NorthGuard fixes Kafka's scaling limits with log striping, sharded metadata, and self-balancing operations.



Industry Media Coverage

- [LinkedIn Created Kafka but It Is Ditching It for Something Better – Analytics India Mag](#)

A broader industry view on why LinkedIn needed to move beyond Kafka.

- [LinkedIn Introduces NorthGuard and XInfra to Replace Kafka – SiliconANGLE](#)

Highlights scale numbers and strategic importance of NorthGuard.

- [Goodbye Kafka: LinkedIn's NorthGuard Ushers in the Future of Stream Processing – Medium \(Sonal Sadafal\)](#)

A narrative-style explanation of how NorthGuard improves durability, scalability, and developer experience.



Coming Up in Future Editions

Here's a preview of upcoming editions, each tackling a critical facet of modern system design

- **Taming Threads and Scaling Smarter**

The art of managing concurrency in the age of multicores and microservices

- **SLAs, SLOs, and the True Measure of Client Experience**

How to set, measure, and align reliability goals across teams

- **Designing for Delight — UX at the Infrastructure Layer**

Why Developer Experience (DX) matters as much as UX

- **Self-Healing Systems**

Building systems that monitor, repair, and recover autonomously

- **Security by Design — Embedding Trust Into Your Architecture**

Proactive defense from the first commit to production

- **The Couch Potato and the Bouncy Bunny — A System Design Story**

A fun, story-driven analogy that explains lazy evaluation vs. eager execution in large-scale systems.

- **Java 21 Virtual Threads — Dude, Where's My Lock?**

Demystifying concurrency bottlenecks and showing how virtual threads change the game for scaling Java apps.

- **Kafka Deep Dives Sessions:**

- **Kafka Storage with In-Sync Replicas (ISR) Explained** — *How ISR ensures data durability and high availability in Kafka clusters*
- **Kafka Consumer Group Protocol Explained** — *The mechanism that keeps consumers coordinated, balanced, and fault-tolerant*
- **Kafka Consumer Lag Explained** — *What lag really means, why it matters, and how to keep it under control*

Which one should I cover first? Tell me in the comments or DM me your vote.



Closing Thought

What started as a fix for brittle XML pipelines turned into Kafka, the backbone of modern data streaming.

Now, LinkedIn is evolving again with NorthGuard. The lesson? **Infrastructure is never “done”—it evolves with scale.**

LinkedIn’s infrastructure evolution demonstrates engineering culture that prioritizes long-term scalability over short-term convenience, even when it means replacing their own open-source contributions to the world.

👉 If you enjoyed this breakdown, follow **Beyond the Stack** for more deep dives—and let me know in the comments which upcoming topic excites you most.